POLER-OS

Windows 10 Registry Deep Analysis

Source: Windows 10 22H2 English x64 (Build 19045.2965)
Hive Files: SYSTEM, SOFTWARE, COMPONENTS, DEFAULT, SAM, SECURITY

Purpose: Extract and analyze Windows registry structure for POLER-OS NT API compatibility layer implementation

2026-07-18

1. Executive Summary

This document presents the results of a deep analysis of the Windows 10 22H2 (Build 19045.2965) registry, extracted directly from the distribution WIM image without installing the operating system. The analysis covers the critical registry hives (SYSTEM, SOFTWARE, COMPONENTS) and extracts structural data essential for implementing the NT API compatibility layer in POLER-OS. The analysis reveals 473 NtXxx system calls that must be implemented, 29 KnownDLLs that must be force-loaded, 93 boot drivers that initialize hardware, and the complete API Set mapping architecture that Windows uses to redirect API calls from virtual api-ms-win-* DLL names to real implementation DLLs. Additionally, the Win32 subsystem registration (csrss.exe + win32k.sys), the service group boot order with 72 ordered groups, and the cryptography provider architecture are documented in detail. This data provides the foundation for POLER-OS to achieve Win32 compatibility at a level deeper than Wine and ReactOS, by directly implementing the registry-driven configuration that Windows itself uses rather than reverse-engineering behavioral patterns.

Key Metrics

Metric	Value	Notes
NtXxx System Calls	473	Core NT syscall table
ZwXxx Kernel Calls	472	Kernel-mode syscall mirrors
Nt/Zw Pairs	472	Near 1:1 correspondence
RtlXxx Runtime Functions	994	Runtime library helpers
KnownDLLs	29	Force-loaded system DLLs
Boot Drivers (Start=0)	93	Hardware initialization
System Drivers (Start=1)	29	IO subsystem init
Total Services/Drivers	663	Complete driver ecosystem
API Set Categories	7	api-ms-win-core, -crt, -service...
API Set DLLs (api-ms-win-*)	91	Virtual DLL redirections
System32 DLLs Total	3,105	Complete DLL inventory
Device Class GUIDs	110	Hardware class categories
Service Boot Groups	72	Ordered initialization sequence
CSP Crypto Providers	10	Cryptographic Service Providers
IFEO Entries	23	Process interception points

2. API Set Mapping Architecture

The API Set mechanism is one of the most critical architectural features that POLER-OS must implement correctly. Starting with Windows 7, Microsoft introduced API Sets as a way to decouple API contracts from implementation DLLs. Instead of applications linking directly to kernel32.dll or advapi32.dll, they link to virtual DLLs with names like api-ms-win-core-file-l1-2-0.dll. The OS then resolves these virtual names to actual implementation DLLs at load time. This indirection layer allows Microsoft to reorganize internal DLL structure without breaking application compatibility. For POLER-OS, implementing this mapping correctly is essential because modern Windows applications and anti-cheat systems expect this redirection mechanism to function properly.

2.1 API Set Resolution Mechanism

The API Set map is NOT stored directly in the offline registry. The key HKLM\SYSTEM\CurrentControlSet\Control\ApiSetMap exists only at runtime and is populated by the kernel from apisetchema.dll during boot. However, the registry does contain ApiSetSchemaExtensions under Session Manager, which adds extensions to the base schema. The actual API Set resolution works through a binary structure inside apisetchema.dll that maps each virtual DLL name prefix to a list of candidate implementation DLLs, with the first valid candidate being selected. In our analysis of the offline WIM image, we found 91 api-ms-win-* DLL files in the System32 directory and additional API set extension data in the Session Manager registry key. The Win10 22H2 distribution uses API Set schema version 6, which supports both api-ms-win-* and ext-ms-win-* namespace prefixes.

2.2 API Set Categories

Category	DLL Count
api-ms-win-core	63
api-ms-win-crt	15
api-ms-win-service	7
api-ms-win-security	3
api-ms-win-base	1
api-ms-win-eventing	1
api-ms-win-shcore	1

The api-ms-win-core category dominates with 63 DLLs, covering file I/O, memory management, process/thread operations, synchronization, registry access, and other fundamental operations. This is the primary target for POLER-OS implementation. The api-ms-win-crt category (15 DLLs) covers the C runtime, which maps primarily to ucrtbase.dll. The api-ms-win-service category (7 DLLs) covers service control manager operations. For POLER-OS, implementing the core category first provides the foundation for running most applications, while the security and service categories are critical for anti-cheat compatibility.

2.3 Complete API Set DLL Inventory

DLL 1	DLL 2	DLL 3
api-ms-win-base-util-l1-1-0.dll	api-ms-win-core-com-l1-1-0.dll	api-ms-win-core-comm-l1-1-0.dll
api-ms-win-core-console-l1-1-0.dll	api-ms-win-core-datetime-l1-1-0.dll	api-ms-win-core-datetime-l1-1-1.dll
api-ms-win-core-debug-l1-1-0.dll	api-ms-win-core-debug-l1-1-1.dll	api-ms-win-core-delayload-l1-1-0.dll
api-ms-win-core-errorhandling-l1-1-0.dll	api-ms-win-core-errorhandling-l1-1-1.dll	api-ms-win-core-fibers-l1-1-0.dll
api-ms-win-core-fibers-l1-1-1.dll	api-ms-win-core-file-l1-1-0.dll	api-ms-win-core-file-l1-2-0.dll
api-ms-win-core-file-l1-2-1.dll	api-ms-win-core-handle-l1-1-0.dll	api-ms-win-core-heap-l1-1-0.dll
api-ms-win-core-interlocked-l1-1-0.dll	api-ms-win-core-io-l1-1-0.dll	api-ms-win-core-io-l1-1-1.dll
api-ms-win-core-kernel32-legacy-l1-1-0.dll	api-ms-win-core-kernel32-legacy-l1-1-1.dll	api-ms-win-core-libraryloader-l1-1-0.dll
api-ms-win-core-libraryloader-l1-1-1.dll	api-ms-win-core-localization-l1-2-0.dll	api-ms-win-core-localization-l1-2-1.dll
api-ms-win-core-memory-l1-1-0.dll	api-ms-win-core-memory-l1-1-1.dll	api-ms-win-core-memory-l1-1-2.dll
api-ms-win-core-namedpipe-l1-1-0.dll	api-ms-win-core-privateprofile-l1-1-0.dll	api-ms-win-core-privateprofile-l1-1-1.dll
api-ms-win-core-processenvironment-l1-1-0.dll	api-ms-win-core-processenvironment-l1-2-0.dll	api-ms-win-core-processthreads-l1-1-0.dll
api-ms-win-core-processthreads-l1-1-1.dll	api-ms-win-core-processthreads-l1-1-2.dll	api-ms-win-core-processtopology-obsolete-l1-1-0.dll
api-ms-win-core-profile-l1-1-0.dll	api-ms-win-core-realtime-l1-1-0.dll	api-ms-win-core-registry-l1-1-0.dll
api-ms-win-core-registry-l2-1-0.dll	api-ms-win-core-rtlsupport-l1-1-0.dll	api-ms-win-core-shlwapi-legacy-l1-1-0.dll
api-ms-win-core-shlwapi-obsolete-l1-1-0.dll	api-ms-win-core-shutdown-l1-1-0.dll	api-ms-win-core-string-l1-1-0.dll
api-ms-win-core-stringansi-l1-1-0.dll	api-ms-win-core-stringloader-l1-1-1.dll	api-ms-win-core-synch-l1-1-0.dll
api-ms-win-core-synch-l1-2-0.dll	api-ms-win-core-sysinfo-l1-1-0.dll	api-ms-win-core-sysinfo-l1-2-0.dll
api-ms-win-core-sysinfo-l1-2-1.dll	api-ms-win-core-threadpool-l1-2-0.dll	api-ms-win-core-threadpool-legacy-l1-1-0.dll
api-ms-win-core-threadpool-private-l1-1-0.dll	api-ms-win-core-timezone-l1-1-0.dll	api-ms-win-core-url-l1-1-0.dll
api-ms-win-core-util-l1-1-0.dll	api-ms-win-core-version-l1-1-0.dll	api-ms-win-core-wow64-l1-1-0.dll
api-ms-win-core-xstate-l1-1-0.dll	api-ms-win-crt-conio-l1-1-0.dll	api-ms-win-crt-convert-l1-1-0.dll
api-ms-win-crt-environment-l1-1-0.dll	api-ms-win-crt-filesystem-l1-1-0.dll	api-ms-win-crt-heap-l1-1-0.dll
api-ms-win-crt-locale-l1-1-0.dll	api-ms-win-crt-math-l1-1-0.dll	api-ms-win-crt-multibyte-l1-1-0.dll
api-ms-win-crt-private-l1-1-0.dll	api-ms-win-crt-process-l1-1-0.dll	api-ms-win-crt-runtime-l1-1-0.dll
api-ms-win-crt-stdio-l1-1-0.dll	api-ms-win-crt-string-l1-1-0.dll	api-ms-win-crt-time-l1-1-0.dll
api-ms-win-crt-utility-l1-1-0.dll	api-ms-win-eventing-consumer-l1-1-0.dll	api-ms-win-security-base-l1-1-0.dll
api-ms-win-security-cryptoapi-l1-1-0.dll	api-ms-win-security-sddl-l1-1-0.dll	api-ms-win-service-core-l1-1-0.dll
api-ms-win-service-core-l1-1-1.dll	api-ms-win-service-management-l1-1-0.dll	api-ms-win-service-management-l2-1-0.dll
api-ms-win-service-private-l1-1-0.dll	api-ms-win-service-private-l1-1-1.dll	api-ms-win-service-winsvc-l1-1-0.dll
api-ms-win-shcore-stream-l1-1-0.dll		

3. KnownDLLs System

KnownDLLs is a security mechanism that forces certain system DLLs to always be loaded from the System32 directory, preventing DLL search order hijacking attacks. When the loader encounters a DLL name that is in the KnownDLLs list, it ignores the application's DLL search path and loads directly from the system directory. This is critical for anti-cheat systems and game integrity, as it prevents attackers from placing malicious versions of system DLLs in application directories. The Session Manager (smss.exe) creates the KnownDLLs object directory during boot, mapping each DLL name to its memory-mapped section. For POLER-OS, this mechanism must be replicated exactly: the kernel must maintain the KnownDLLs object directory and the loader must check it before searching the normal DLL path.

Internal Name	DLL File	Type
COMDLG32	COMDLG32.dll	1
DifxApi	difxapi.dll	1
IMAGEHLP	IMAGEHLP.dll	1
IMM32	IMM32.dll	1
MSCTF	MSCTF.dll	1
MSVCRT	MSVCRT.dll	1
NORMALIZ	NORMALIZ.dll	1
NSI	NSI.dll	1
OLEAUT32	OLEAUT32.dll	1
PSAPI	PSAPI.DLL	1
SHCORE	SHCORE.dll	1
SHELL32	SHELL32.dll	1
SHLWAPI	SHLWAPI.dll	1
Setupapi	Setupapi.dll	1
WLDAP32	WLDAP32.dll	1
WS2_32	WS2_32.dll	1
advapi32	advapi32.dll	1
clbcatq	clbcatq.dll	1
combase	combase.dll	1
coml2	coml2.dll	1
gdi32	gdi32.dll	1
gdiplus	gdiplus.dll	1
kernel32	kernel32.dll	1
ole32	ole32.dll	1
rpcrt4	rpcrt4.dll	1
sechost	sechost.dll	1
user32	user32.dll	1
wow64	wow64.dll	1
wow64win	wow64win.dll	1

4. NT System Call Table (ntdll.dll)

The ntdll.dll export table is the definitive interface between user-mode applications and the NT kernel. Every Win32 API call eventually resolves to one or more NtXxx functions in ntdll.dll, which transition to kernel mode via the syscall instruction. The ZwXxx functions are identical to NtXxx but are intended for kernel-mode callers - they skip the previous-mode check that NtXxx performs. In Win10 22H2, ntdll.dll exports 473 NtXxx functions and 472 ZwXxx functions, with 472 Nt/Zw pairs. This near-perfect pairing means that POLER-OS must implement a syscall table with approximately 473 entries. Additionally, there are 994 RtlXxx runtime library functions that provide user-mode utilities like string manipulation, security descriptor operations, and memory management. These Rtl functions do NOT transition to kernel mode - they are pure user-mode code that POLER-OS must implement in its ntdll compatibility layer.

4.1 Syscall Classification

Category	Count	Examples
Other	141	NtAddAtom, NtAddAtomEx, NtAddBootEntry, NtAddDriverEntry, NtAllocateLocallyUniqueId...
Synchronization	62	NtAcquireCrossVmMutant, NtAlpcSendWaitReceivePort, NtAssociateWaitCompletionPacket, NtCancelTimer, NtCancelTimer2...
Registry	49	NtCommitRegistryTransaction, NtCompactKeys, NtCompressKey, NtCreateKey, NtCreateKeyTransacted...
Process/Thread	47	NtAcquireProcessActivityReference, NtAlertResumeThread, NtAlertThread, NtAlertThreadByThreadId, NtAlpcOpenSenderProcess...
File/IO	44	NtAreMappedFilesTheSame, NtCancelIoFile, NtCancelIoFileEx, NtCancelSynchronousIoFile, NtCopyFileChunk...
IPC/Port	43	NtAcceptConnectPort, NtAlpcAcceptConnectPort, NtAlpcCancelMessage, NtAlpcConnectPort, NtAlpcConnectPortEx...
Object Manager	38	NtAllocateReserveObject, NtAssignProcessToJobObject, NtCloseObjectAuditAlarm, NtCompareObjects, NtCreateDebugObject...
Transaction/TM	36	NtCommitEnlistment, NtCommitRegistryTransaction, NtCommitTransaction, NtCreateEnlistment, NtCreateRegistryTransaction...
Security/Token	34	NtAccessCheck, NtAccessCheckAndAuditAlarm, NtAccessCheckByType, NtAccessCheckByTypeAndAuditAlarm, NtAccessCheckByTypeResultList...
Memory/VM	25	NtAllocateVirtualMemory, NtAllocateVirtualMemoryEx, NtAlpcCreatePortSection, NtAlpcCreateSectionView, NtAlpcDeletePortSection...

The largest category is Process/Thread operations, which includes process creation (NtCreateProcess, NtCreateUserProcess), thread management (NtCreateThreadEx, NtSetInformationThread), and job object control. This is the primary attack surface for anti-cheat systems, which monitor process and thread creation through kernel callbacks. The File/IO category covers all file operations that POLER-FS must support, including NtCreateFile, NtReadFile, NtWriteFile, and NtDeviceIoControlFile. The IPC/Port category includes ALPC (Advanced Local Procedure Call) which is the primary inter-process communication mechanism in NT and is heavily used by both the CSRSS subsystem and anti-cheat components.

5. Boot Sequence & Service Group Order

The Windows boot process follows a strictly ordered sequence defined by the ServiceGroupOrder registry key. This key contains a list of 72 driver groups that must be loaded in a specific order. Each driver in the Services key has a "Group" value that places it in one of these groups, and a "Tag" value that determines its order within the group. Understanding this sequence is critical for POLER-OS because the kernel must initialize drivers in the same order to ensure that dependencies are satisfied. For example, the ACPI driver must load before PCI, which must load before storage drivers, which must load before the filesystem. Getting this order wrong results in bluescreens or non-functional hardware. The 93 boot drivers (Start=0) are loaded by the boot loader (winload.exe) before the kernel even starts, while the 29 system drivers (Start=1) are loaded by IoInitSystem during kernel initialization.

5.1 Service Group Order (First 40 Groups)

#	Group Name
1	System Reserved
2	EMS
3	WdfLoadGroup
4	Boot Bus Extender
5	System Bus Extender
6	SCSI miniport
7	Port
8	Primary Disk
9	SCSI Class
10	SCSI CDROM Class
11	FSFilter Infrastructure

#	Group Name
12	FSFilter System
13	FSFilter Bottom
14	FSFilter Copy Protection
15	FSFilter Security Enhancer
16	FSFilter Open File
17	FSFilter Physical Quota Management
18	FSFilter Virtualization
19	FSFilter Encryption
20	FSFilter Compression
21	FSFilter Imaging
22	FSFilter HSM
23	FSFilter Cluster File System
24	FSFilter System Recovery
25	FSFilter Quota Management
26	FSFilter Content Screener
27	FSFilter Continuous Backup
28	FSFilter Replication
29	FSFilter Anti-Virus
30	FSFilter Undelete
31	FSFilter Activity Monitor
32	FSFilter Top
33	Filter
34	Boot File System
35	Base
36	Pointer Port
37	Keyboard Port
38	Pointer Class
39	Keyboard Class
40	Video Init

5.2 Boot Drivers by Group

Group	Count	Drivers

6. Win32 Subsystem Configuration

The Win32 subsystem is the primary user-mode subsystem in Windows NT. Its configuration is stored in the registry under HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\SubSystems and defines how the Windows subsystem is initialized. The "Windows" value specifies the command line for CSRSS (Client/Server Runtime Subsystem), which is the first user-mode process created by the kernel. The "Required" value lists subsystems that must start successfully for boot to continue, while "Optional" lists subsystems that can fail without preventing boot. The "Kmode" value specifies the kernel-mode portion of the subsystem (win32k.sys), which implements the window manager and GDI. For POLER-OS, this configuration is essential because it defines the exact process by which the Win32 environment is brought online, including the server DLLs (basesrv, winsrv, sxssrv) that CSRSS loads and the shared section parameters that control desktop heap allocation.

Value	Data
(default)	mnmsrvc
Debug	
Kmode	\SystemRoot\System32\win32k.sys

Value	Data
Optional	['']
Required	['Debug', 'Windows', '', '']
Windows	%SystemRoot%\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,20480,768 Windows=On SubSystemType=Windows ServerDll=basesrv,1 ServerDll=w...

6.1 CSRSS Command Line Breakdown

Parameter	Value	Description
ObjectDirectory	\Windows	NT object directory for Win32 objects
SharedSection	1024,20480,768	Desktop heap sizes: shared, interactive, non-interactive (KB)
Windows	On	Enable windowing subsystem
SubSystemType	Windows	Subsystem type identifier
ServerDll	basesrv,1	Base server DLL (console, temp files)
ServerDll	winsrv:UserServerDllInitialization, 3	Window manager server DLL
ServerDll	sxssrv,4	SxS (side-by-side) server DLL
ProfileControl	Off	Profiling disabled
MaxRequestThreads	16	Maximum CSRSS worker threads

7. Session Manager Configuration

The Session Manager (smss.exe) is the first user-mode process created by the NT kernel. It is responsible for creating the NT object namespace, initializing the KnownDLLs object directory, setting up DOS device mappings, starting the Win32 subsystem (CSRSS), and creating the initial Windows session. Its configuration in the registry under HKLM\SYSTEM\CurrentControlSet\Control\Session Manager defines every aspect of the early boot environment. For POLER-OS, this data is critical because the Session Manager must be one of the first user-mode components implemented, and it must configure the object namespace and KnownDLLs exactly as Windows does to ensure compatibility with applications and anti-cheat systems that depend on these namespace conventions.

7.1 DOS Device Mappings

These mappings define the NT device namespace that Win32 applications see. They are created by the Session Manager during boot and map DOS-style device names to NT device objects. POLER-OS must create identical symbolic links in its object manager namespace for Win32 compatibility.

DOS Name	NT Device Path
AUX	\DosDevices\COM1
CON	\Device\ConDrv\Console
CONIN$	\Device\ConDrv\CurrentIn
CONOUT$	\Device\ConDrv\CurrentOut
MAILSLOT	\Device\MailSlot
NUL	\Device\Null
PIPE	\Device\NamedPipe
PRN	\DosDevices\LPT1
Silos	\Silos
UNC	\Device\Mup

7.2 Memory Management Parameters

Parameter	Value
ClearPageFileAtShutdown	0
DisablePagingExecutive	0

Parameter	Value
LargeSystemCache	0
NonPagedPoolQuota	0
NonPagedPoolSize	0
PagedPoolQuota	0
PagedPoolSize	0
PagingFiles	['?:\\pagefile.sys', '', '']
SecondLevelDataCache	0
SessionPoolSize	4
SessionViewSize	48
SystemPages	0

8. Cryptography Provider Architecture

The Windows Cryptography architecture uses Cryptographic Service Providers (CSPs) that implement various cryptographic algorithms through a plugin system. The registry under HKLM\SOFTWARE\Microsoft\Cryptography defines the available providers, their types, and the algorithms they support. This is critically important for POLER-OS because anti-cheat systems use the Windows cryptography APIs to verify game integrity and authenticate the game client. If the crypto provider architecture is not implemented correctly, anti-cheat signature verification will fail and the game will be rejected. POLER-OS must implement at minimum the Microsoft Enhanced RSA and AES Cryptographic Provider and the Microsoft Strong Cryptographic Provider, along with CNG (Cryptography Next Generation) providers for modern applications.

8.1 Cryptographic Service Providers

Provider Name
Microsoft Base Cryptographic Provider v1.0
Microsoft Base DSS and Diffie-Hellman Cryptographic Provider
Microsoft Base DSS Cryptographic Provider
Microsoft Base Smart Card Crypto Provider
Microsoft DH SChannel Cryptographic Provider
Microsoft Enhanced Cryptographic Provider v1.0
Microsoft Enhanced DSS and Diffie-Hellman Cryptographic Provider
Microsoft Enhanced RSA and AES Cryptographic Provider
Microsoft RSA SChannel Cryptographic Provider
Microsoft Strong Cryptographic Provider

8.2 Provider Types

Type	Description
Type 001	RSA Full (Signature + Key Exchange)
Type 003	DSS Signature (DSA)
Type 012	RSA SChannel (SSL/TLS)
Type 013	DSS Signature + Diffie-Hellman
Type 018	Enhanced RSA and AES
Type 024	CNG (Cryptography Next Generation)

9. Device Class GUIDs

Windows organizes hardware devices into device classes, each identified by a GUID. These class GUIDs are used throughout the system for driver matching, device installation, and security policy enforcement. The registry key HKLM\SYSTEM\CurrentControlSet\Control\Class contains 110 device class entries, each with properties like the class

name, installer DLL, and icon resource. For POLER-OS, these GUIDs must be replicated in the device enumeration subsystem so that applications and drivers that query device class information receive the expected data. This is particularly important for anti-cheat systems that check hardware device properties to detect virtualization or cheating tools.

GUID	Class Name
{05f5cfe2-4733-4950-a6bb-07aad01a3a84}	XboxComposite
{13e42dfa-85d9-424d-8646-28a70f864f9c}	RemotePosDevice
{14b62f50-3f15-11dd-ae16-0800200c9a66}	DigitalMediaDevices
{1ed2bbf9-11f0-4084-b21f-ad83a8e6dcdc}	PrintQueue
{25dbce51-6c8f-4a72-8a6d-b54c2b4fc835}	WCEUSBS
{268c95a1-edfe-11d3-95c3-0010dc4050a5}	SecurityAccelerator
{2a9fe532-0cdc-44f9-9827-76192f2ca2fb}	HidMsr
{2db15374-706e-4131-a0c7-d7c78eb0289a}	SystemRecovery
{36fc9e60-c465-11cf-8056-444553540000}	USB
{3e3f0674-c83c-4558-bb26-9820e1eba5c5}	ContentScreener
{43675d81-502a-4a82-9f84-b75f418c5dea}	Media Center Extender
{4658ee7e-f050-11d1-b6bd-00c04fa372a7}	PnpPrinters
{48721b56-6795-11d2-b1a8-0080c72e74a2}	Dot4
{48d3ebc4-4cf8-48ff-b869-9c68ad42eb9f}	Replication
{49ce6ac8-6f86-11d2-b1e5-0080c72e74a2}	Dot4Print
{4d36e965-e325-11ce-bfc1-08002be10318}	CDROM
{4d36e966-e325-11ce-bfc1-08002be10318}	Computer
{4d36e967-e325-11ce-bfc1-08002be10318}	DiskDrive
{4d36e968-e325-11ce-bfc1-08002be10318}	Display
{4d36e969-e325-11ce-bfc1-08002be10318}	FDC
{4d36e96a-e325-11ce-bfc1-08002be10318}	HDC
{4d36e96b-e325-11ce-bfc1-08002be10318}	Keyboard
{4d36e96c-e325-11ce-bfc1-08002be10318}	MEDIA
{4d36e96d-e325-11ce-bfc1-08002be10318}	Modem
{4d36e96e-e325-11ce-bfc1-08002be10318}	Monitor
{4d36e96f-e325-11ce-bfc1-08002be10318}	Mouse
{4d36e970-e325-11ce-bfc1-08002be10318}	MTD
{4d36e971-e325-11ce-bfc1-08002be10318}	MultiFunction
{4d36e972-e325-11ce-bfc1-08002be10318}	Net
{4d36e973-e325-11ce-bfc1-08002be10318}	NetClient

... and 80 more device classes

10. Image File Execution Options (IFEO)

The Image File Execution Options registry key allows the system to intercept and modify process launch behavior. This is a dual-use mechanism: legitimate debuggers use it to attach to processes, while anti-cheat systems use it to inject monitoring DLLs into game processes. The key exists at HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options and contains subkeys named after executable files. Each subkey can specify values like Debugger (redirects process launch to a debugger), MitigationOptions (enables/disables exploit mitigations), and DisableExceptionChainValidation (modifies exception handling behavior). In the clean Windows 10 installation, we found 23 IFEO entries, primarily for Internet Explorer and system utilities. Anti-cheat systems like EasyAntiCheat and BattlEye add their own IFEO entries at install time. POLER-OS must implement IFEO support to allow anti-cheat DLLs to inject into game processes, as this is one of the primary methods they use for monitoring.

Executable	Options
ExtExport.exe	MitigationOptions=256
ie4uinit.exe	MitigationOptions=256

Executable	Options
ieinstal.exe	MitigationOptions=256
ielowutil.exe	MitigationOptions=256
ieUnatt.exe	MitigationOptions=256
iexplore.exe	DisableExceptionChainValidation=0, DisableUserModeCallbackFilter=1, MitigationOptions=256
MicrosoftEdgeUpdate.exe	DisableExceptionChainValidation=0
MRT.exe	CFGOptions=1
mscorsvw.exe	MitigationOptions=4294967296
msfeedssync.exe	MitigationOptions=256
mshta.exe	MitigationOptions=256
MsMpEng.exe	CFGOptions=1
MsSense.exe	MitigationOptions=16777232
ngen.exe	MitigationOptions=4294967296
ngentask.exe	MitigationOptions=4294967296
PresentationHost.exe	MitigationOptions=1118481
PrintDialog.exe	MitigationOptions=4294967296
PrintIsolationHost.exe	MitigationOptions=2097152
runtimebroker.exe	MitigationOptions=4294967296
splwow64.exe	MitigationOptions=2097152
spoolsv.exe	MitigationOptions=2097152
svchost.exe	MinimumStackCommitInBytes=32768
SystemSettings.exe	MitigationOptions=4294967296

11. File System Parameters

The file system configuration in HKLM\SYSTEM\CurrentControlSet\Control\FileSystem defines how NTFS and ReFS behave at the kernel level. These parameters control features like 8.3 name generation, last access time updates, encryption, compression, and symlink evaluation. For POLER-OS, these parameters directly inform the POLER-FS implementation. The most significant finding is that NtfsDisableLastAccessUpdate is set to 1 by default, meaning NTFS no longer updates the last access timestamp on reads, which is a significant performance optimization that POLER-FS should replicate. Additionally, NtfsDisable8dot3NameCreation is set to 2 (volume-dependent), meaning short name generation is optional and should be disabled on the system volume for performance. Symlink evaluation settings show that local-to-local and local-to-remote symlinks are enabled by default, while remote-to-local and remote-to-remote are disabled for security.

Parameter	Value
DisableDeleteNotification	0
FilterSupportedFeaturesMode	0
LongPathsEnabled	0
NtfsAllowExtendedCharacter8dot3Rename	0
NtfsBugcheckOnCorrupt	0
NtfsDisable8dot3NameCreation	2
NtfsDisableCompression	0
NtfsDisableEncryption	0
NtfsDisableLastAccessUpdate	1
NtfsDisableLfsDowngrade	0
NtfsDisableVolsnapHints	0
NtfsEncryptPagingFile	0
NtfsMemoryUsage	0
NtfsMftZoneReservation	0
NtfsQuotaNotifyRate	3600

Parameter	Value
RefsDisableLastAccessUpdate	1
ScrubMode	2
SymlinkLocalToLocalEvaluation	1
SymlinkLocalToRemoteEvaluation	1
SymlinkRemoteToLocalEvaluation	0
SymlinkRemoteToRemoteEvaluation	0
UdfsCloseSessionOnEject	3
UdfsSoftwareDefectManagement	0
Win31FileSystem	0
Win95TruncatedExtensions	1
SuppressInheritanceSupport	1

12. Local Security Authority (LSA)

The LSA configuration defines authentication packages, notification packages, and security packages. The authentication package msv1_0 handles NTLM authentication, while the notification package scecli handles security policy updates. The Security Packages list defines which SSP (Security Support Provider) DLLs are loaded into the LSA process. In the clean install, this is empty (populated at runtime with kerberos, msv1_0, schannel, wdigest, tspkg, pku2u, and cloudAP). For POLER-OS, the LSA architecture must be implemented because anti-cheat systems use the LSA authentication APIs to verify the integrity of the logon session and detect token manipulation. The NoLmHash=1 setting indicates that LM hash storage is disabled, which is the modern security default that POLER-OS should follow. The LimitBlankPasswordUse=1 setting prevents remote authentication with blank passwords, another security default to replicate.

Parameter	Value
auditbasedirectories	0
auditbaseobjects	0
Bounds	■
crashonauditfail	0
fullprivilegeauditing	0
LimitBlankPasswordUse	1
NoLmHash	1
Security Packages	['""', '', '']
Notification Packages	['scecli', '', '']
Authentication Packages	['msv1_0', '', '']

13. Windows NT Version Information

The Windows NT CurrentVersion key contains the operating system version information that applications query to determine compatibility. The CurrentVersion value is 6.3 (the internal NT version for Windows 10), while the CurrentBuild is 19045 (the public build number). The BuildLab value (19041.vb_release.191206-1406) identifies the exact build configuration. The UBR (Update Build Revision) value of 2965 identifies the specific cumulative update level. For POLER-OS, these values must be spoofed correctly because applications and anti-cheat systems check the OS version to determine feature availability and compatibility. Reporting an incorrect version can cause applications to refuse to run or to use incompatible code paths.

Key	Value
CurrentVersion	6.3
CurrentBuild	19045
CurrentBuildNumber	19045
BuildLab	19041.vb_release.191206-1406
BuildLabEx	19041.1.amd64fre.vb_release.191206-1406
CurrentType	Multiprocessor Checked

Key	Value
InstallationType	Client
EditionID	Professional
ProductName	Windows 10 Enterprise
UBR	2965
PathName	X:\Windows
SystemRoot	X:\Windows
DigitalProductId	¤■■■■■■■■■■■■■■■7■■■■■■■■■1■■■■■■■■ ■■...

14. POLER-OS Implementation Priority

Based on the registry analysis, the following implementation priority is recommended for the POLER-OS NT API compatibility layer. The priorities are ordered by the impact on Win32 application compatibility and anti-cheat support. Each phase builds on the previous one, and skipping phases will result in applications failing to load or crashing at runtime.

Priority	Subsystem	Key Functions	Rationale
P0	Object Manager	Namespace, symlinks, DOS devices, KnownDLLs directory	All apps depend on object namespace
P0	Process/Thread	NtCreateProcess, NtCreateThreadEx, NtTerminateProcess	No app can run without this
P0	Memory Manager	NtAllocateVirtualMemory, NtProtectVirtualMemory, sections	Fundamental for all code execution
P0	File System	NtCreateFile, NtReadFile, NtWriteFile, POLER-FS integration	No I/O without this
P1	API Set Layer	api-ms-win-* DLL redirection, apisetchema.dll parser	Modern apps won't load without it
P1	Registry	NtCreateKey, NtSetValueKey, NtQueryValueKey, hive format	All configuration depends on registry
P1	Synchronization	NtCreateEvent, NtCreateMutant, NtWaitForSingleObject	Multi-threading impossible without it
P1	KnownDLLs	Force-load 29 system DLLs from System32 only	Security requirement for anti-cheat
P2	IPC/ALPC	NtCreatePort, NtConnectPort, NtRequestWaitReplyPort	Required for CSRSS and service communication
P2	Security/Token	NtOpenProcessToken, NtAccessCheck, LSA integration	Anti-cheat authentication depends on this
P2	Win32k	win32k.sys kernel-mode GDI/Window manager	Required for any GUI application
P2	CSP/CNG	Crypto providers, bcrypt, ncrypt	Anti-cheat signature verification
P3	IFEO	Process interception, MitigationOptions	Anti-cheat DLL injection mechanism
P3	WMI	WMI provider hosting, ETW tracing	Anti-cheat monitoring infrastructure
P3	Device Classes	GUID enumeration, device installation	Hardware detection and anti-cheat checks

15. Winlogon Configuration

Winlogon is responsible for interactive logon, shell startup, and screen saver security. Its configuration defines the shell (explorer.exe), user initialization (userinit.exe), and various security policies. For POLER-OS, the Winlogon configuration is important because it defines the user-mode entry point after kernel initialization. The Shell value specifies the application that Winlogon starts after authentication - by default this is explorer.exe, but POLER-OS could replace this with its own shell. The Userinit value specifies the program that initializes the user environment before the shell starts. The AutoRestartShell=1 setting means that if the shell crashes, Winlogon will automatically restart it, which is important for system stability.

Parameter	Value
AutoRestartShell	1
Background	0 0 0

Parameter	Value
CachedLogonsCount	10
DebugServerCommand	no
DefaultDomainName	
DefaultUserName	
DisableBackButton	1
EnableSIHostIntegration	1
ForceUnlockLogon	0
LegalNoticeCaption	
LegalNoticeText	
PasswordExpiryWarning	5
PowerdownAfterShutdown	0
PreCreateKnownFolders	{A520A1A4-1780-4FF6-BD18-167343C5AF16}
ReportBootOk	1